

零样本模型在客户真实数据上的效果评估

INST623 AI Adoption Clinic — Final Project 实验人：Chihao Li (Technical Lead)

实验日期：2026-04-12

0. TL;DR

1. 本次把 **CLIP ViT-B/32** 和 Google 刚开源 (2026-04) 的 **Gemma 4 E4B-IT** (4-bit 量化, MLX) 两个零样本模型, 拉到客户 98 张真实 inspector 照片 + 9 个 Riverdale Park 法条类别上做评估。两个模型都不吃任何标注数据, 不微调。
2. 最终结果:**Gemma 4 E4B-IT 在 multi-label 评估下拿到 top-1 85.7%、top-3 96.9%**; CLIP 在同评估下是 top-1 80.6%、top-3 94.9%。Gemma 明确反超 CLIP 约 5 个点。
3. 但**比数字更重要的发现是**: 在做完错误归因之后我们意识到这个任务的框架本身错了——“这张图属于哪一类”不是真正的问题, “对某个特定 code, 这张图能不能支持 cite”才是。前者是 multi-class classification, 后者是 **per-class binary detection**。这个 reframe 把前面所有“失败”重新解释了一遍。
4. 基于 reframe, 我们设计了一个新的**两阶段 pipeline**: CLIP 做 candidate 打分, Gemma 对每个候选做独立的 yes/no 判别。这条路线在数据量少、多违规共存、需要解释性、需要低成本扩类别的场景下, 很可能比训练 DINOv2 更合适, 而且**不 block 客户数据到齐**。
5. 唯一真正的 blocker 是**负样本**: 98 张全是正例, 没有“去过现场但判定 no violation”的照片, 所以现在连 precision 都无法严格评估。给 Ryan 的下一轮 data ask 需要明确改成“我们要的不是更多违规照片, 是正常房屋的照片”。

1. 背景：从上周的 proxy 实验到本周的客户数据

上周的实验 (见 notebooks/experiment.ipynb) 在 5 个 proxy 类上对比了 CLIP zero-shot、DINOv2 LP-FT、EfficientNetV2 LP-FT, 数据来自 BD3 / TACO / Grass-Weeds / Aerial Dumping 四个公开数据集, 共约 8,900 张图。那一轮解决了“无客户数据时如何做模型选型”的问题, 但没有回答客户真正关心的问题。

本周 Ryan 发来了第一批真实 inspector 照片, 按官方法条编码组织成 14 个文件夹。这批数据带来三个根本变化:

1. **类别空间完全变了**。客户 taxonomy 比 proxy 5 类细得多, 大部分类别 (boarded_windows、broken_windows、graffiti、chimney、roof shingles、address numbers、inoperable vehicles 等) 在公开数据集里完全没有对应的训练集。
2. **每类样本非常少**。98 张图散落在 9 个有图的文件夹里, 单类最少 3 张, 最多 15 张。这个量级对“训练一个 14 类分类器”来说完全不够。

3. 上周的三个模型里只有 **CLIP 可用**。DINOv2 和 EfficientNetV2 都绑定了上周的 5 个 proxy 类，重训需要新标注 —— 直接堵死。CLIP zero-shot 因为不绑定 taxonomy，是唯一能立刻迁移过来的模型。

所以本周实验的范围严格限定在**零样本路径**：CLIP 延续 Day 1 baseline，加上刚开源的 Gemma 4 E4B（instruction-tuned, 4-bit）作为第二个对照。目的是在不依赖标注数据的前提下，看两个 foundation model 能把客户真实问题解到什么程度，并且给团队后续（ensemble、few-shot、prompt 调优、DINO 重训）一个明确的 baseline。

2. 实验设置

2.1 数据

98 张 inspector 照片，9 个有效类别：

类别	图片数
peeling_paint	15
overgrown_vegetation	14
inoperable_vehicle	14
junk_trash_accumulation	13
broken_windows	12
graffiti	12
boarded_windows	8
damaged_roof_shingles	7
deteriorating_chimney	3
合计	98

其余 5 类（porch / fence / address_numbers / siding / overgrown_trees）本批数据为空，评估中跳过。

2.2 模型

模型	参数量	磁盘	单图耗时	部署方式
CLIP ViT-B/32 (LAION-2B)	~150M	~600 MB	<10 ms (batched)	open_clip + MPS
Gemma 4 E4B-IT (4-bit)	4.5B eff.	~4 GB	~2.2 s	mlx-vlm + Metal

两个模型都在本地 M4 24 GB Mac 上跑。Gemma 选 E4B 是因为 Google 官方推荐它作为“edge / consumer”档位，4-bit 量化后活跃内存 <6 GB，完全不挤压系统。26B A4B 和 31B 档虽然在 24GB 上可跑，但对 98 张图的 benchmark 性价比不划算。

2.3 零样本推理策略

CLIP：每类 3 条自然语言 prompt（比如 "a photo of peeling and chipping paint on a building exterior"），text encoder 编码后各自 L2 normalize，再按类取平均作为 class prototype；图片过 image encoder 后和 9 个 prototype 做 cosine similarity，乘温度 100，softmax，argmax。全过程无训练。

Gemma 4：构造结构化 prompt，把 9 个类 + 视觉描述列给模型，让它针对每张图返回一个 one-line JSON：

```
{"ranked": ["peeling_paint", "broken_windows", "boarded_windows"]}
```

正则抽 JSON，把 ranked 列表转成 reciprocal-rank 伪概率 ($1/\text{rank} \rightarrow \text{softmax}$)，让 top-k accuracy、macro F1 等指标可以和 CLIP 可比。temperature=0.0。

2.4 工程踩坑（非常值得记录）

坑 1：加载了 base checkpoint 而不是 instruction-tuned checkpoint

第一次跑 Gemma 加载的是 mlx-community/gemma-4-e4b-4bit (base 模型)，结果是 98 张图的输出全是 "in response to your message, you said..." 这种循环废话。JSON parse 全部失败，fallback 到 class 0，最终 100% 预测 boarded_windows。表面看起来像“模型全废了”，实际是 base 模型根本不会跟 instruction。切到 -it-4bit (instruction-tuned) 之后立刻恢复正常。

教训：多模态 VLM 的 -it 和 base 变体必须分清楚，每次部署前用一个 smoke test 检查 raw output 格式是否符合 parser 预期。

坑 2：nbconvert 黑箱，看不到单张图的进度

jupyter nbconvert --execute 会把整个 cell 的 stdout 憋到 cell 结束才打印，Gemma 跑 98 张图的过程中我完全看不到进度。这既不方便监控，也让失败诊断变得很痛苦。

解法：把 Gemma 推理从 notebook 里抽出来做成独立 Python 脚本，每张图的结果立刻 append 到一个 JSONL 流式日志并 flush。这样另一个终端 tail -f 就能实时看进度，跑完之后 notebook 只是读 cache。

坑 3：parser 有一个隐蔽 bug（见下面 §5 错误归因）

Gemma 偶尔会在 ranked 列表里返回重复类别（比如 ["broken_windows", "boarded_windows", "broken_windows"]）。我原本的 _ranked_to_probs 用后一次出现覆盖前一次出现的分数（0.33 覆盖了 1.0），导致 broken_windows 最终分数反而比 boarded_windows 低，top-1 错判。这个 bug 一个人独占了 Gemma 6 个错判。修复是一行代码（dedupe 时保留第一次出现），修完之后 Gemma top-1 从 68.4% 直接跳到 74.5%。

3. 三组关键数字

我们最终报了三个层次的评估，每一层都在前一层之上加一次修正：

3.1 原始数字（按 label-row 严格比对 folder name）

模型	Top-1	Top-3	Macro F1
CLIP ViT-B/32	69.4%	92.9%	0.678
Gemma 4 E4B-IT（parser 有 bug）	68.4%	92.9%	0.661

这是第一次跑出来的结果 —— 两个模型看起来打平手，Gemma 甚至略低。

3.2 修复 parser bug 后

模型	Top-1	Top-3	Macro F1
CLIP ViT-B/32	69.4%	92.9%	0.678
Gemma 4 E4B-IT（parser 修复）	74.5%	92.9%	0.703

Gemma 一下子反超 CLIP 5 个点。这 5 个点完全不是模型变强，只是我之前错算了。

3.3 Multi-label ground truth（按物理图像的所有 valid labels）

我们发现客户数据里有 **11 张物理照片**（md5 验证）被 client 以完全相同的字节复制到多个类别文件夹下。比如一张 37739844075_42937e37bc_o.jpg **同时出现在 Graffiti、Long Grass、Damaged Roof Shingles 三个文件夹里** —— 客户自己就把这张图打了 3 个标签。

这意味着客户数据的真实结构是 **multi-label**，而不是 single-label。如果 Gemma 预测某张图是“inoperable_vehicle”但这张图的 valid labels 里同时有 inoperable_vehicle 和 overgrown_vegetation，那 Gemma 其实是对的，不是错的。按物理图像的 multi-label ground truth 重新评估：

模型	Strict Top-1	ML Top-1	Strict Top-3	ML Top-3
CLIP ViT-B/32	69.4%	80.6%	92.9%	94.9%
Gemma 4 E4B-IT	74.5%	85.7%	92.9%	96.9%

ML = multi-label aware。Gemma 在真正公平的 ground truth 下达到 **top-1 85.7% / top-3 96.9%**。

4. 分类别表现

4.1 CLIP ViT-B/32

	precision	recall	f1-score	support
boarded_windows	0.400	1.000	0.571	8
broken_windows	0.846	0.917	0.880	12

damaged_roof_shingles	0.750	0.429	0.545	7
deteriorating_chimney	1.000	1.000	1.000	3
graffiti	0.750	1.000	0.857	12
inoperable_vehicle	0.583	1.000	0.737	14
junk_trash_accumulation	0.917	0.846	0.880	13
overgrown_vegetation	1.000	0.071	0.133	14
peeling_paint	1.000	0.333	0.500	15

• **CLIP 擅长的：**chimney (3/3), graffiti (12/12), inoperable_vehicle (14/14), broken_windows (11/12), junk_trash (11/13)。共同特点是视觉主体明确、单一主体——网上的 image-caption 对已经为这些概念建立了很强的先验。

• **CLIP 大跪的两类：**

- overgrown_vegetation recall 7% (1/14)：长草照片里几乎每张都同时有弃车/垃圾/涂鸦，CLIP 选了视觉更突出的主体。
- peeling_paint recall 33% (5/15)：6 张被判成 boarded_windows。patch level 上，风化木质外墙 + 脱落油漆 和 钉死的 plywood 板视觉特征重叠大，ViT-B/32 的 32×32 patch 分辨率不够分开。

4.2 Gemma 4 E4B-IT (parser 修复后)

	precision	recall	f1-score	support
boarded_windows	1.000	0.250	0.400	8
broken_windows	0.909	0.833	0.870	12
damaged_roof_shingles	0.545	0.857	0.667	7
deteriorating_chimney	1.000	0.667	0.800	3
graffiti	0.800	1.000	0.889	12
inoperable_vehicle	0.636	1.000	0.778	14
junk_trash_accumulation	0.857	0.923	0.889	13
overgrown_vegetation	0.500	0.143	0.222	14
peeling_paint	0.765	0.867	0.812	15

Gemma 修好的 CLIP 短板：- peeling_paint recall 从 33% → **87%** (13/15)。Gemma 做的是场景级推理，不会把剥落油漆和 plywood 板混为一谈。- damaged_roof_shingles recall 从 43% → **86%** (6/7)。- junk_trash recall 从 85% → **92%**。

Gemma 新出现的短板：- boarded_windows recall 从 100% → **25%** (2/8)。precision 反而变成 1.0（所有它说是 boarded_windows 的图都真是）。这是一个典型的“model over-specificity”错误：Gemma 看到一栋封板的老房子，注意到剥落油漆、屋顶破损、整体破败，然后从 9 个选项里挑了更具体的子问题（peeling_paint / damaged_roof_shingles），而不是更泛的 boarded_windows。这是 **reasoning error**，不是 **perception error**。

两个模型都跪的：overgrown_vegetation。CLIP 7%、Gemma 21%。失败原因相同——照片里多违规共存。

5. 深度错误归因：Gemma 的 31 个”错”逐个拆

我们原始 Gemma 有 31 个与 label row 不符的预测。把这 31 个逐个过一遍、拆到具体原因，得到一个非常有启发性的分布：

归因	数量	性质
Parser bug (ranked 列表里类重复，后出现覆盖前出现分数)	6	我代码的锅，已修
Multi-label 数据 (物理图像在多个文件夹里，Gemma 预测的是另一个合法标签)	11	客户数据结构问题，multi-label 评估后判对
Ranking error (truth 在 top-3 但 rank 2 或 3)	11	top-3 交付场景下 operationally 无损
真正的 genuine misread (top-3 完全不含任何 valid label)	3	需要单图分析

对 3 个 genuine miss 做了逐图 visual inspection：

图 1：1635092396_5c6b58bf2d_o.jpg (标 boarded_windows)

红色两层老房子。板子实际上钉在门廊栏杆上，不是窗户；二楼窗户完好无损；整张图最醒目的问题是屋顶严重破损和外墙油漆大面积剥落。Gemma 给的 top-3 是 [damaged_roof_shingles, peeling_paint, junk_trash_accumulation] —— 视觉上三个都对。是客户把一张”门廊封板”的照片归进了 § 304.13 “Boarded Windows” 文件夹。这是客户标注边界问题，不是 Gemma 错了。

图 2：4861716269_6e5f0fc8c0_o.jpg (标 boarded_windows)

两栋房子。左边黄房子有一扇小凸窗被钉了板，尺寸和色块都不突出；图里没有明显杂草。Gemma 给的 top-3 是 [overgrown_vegetation, peeling_paint, junk_trash_accumulation]。完全站不住脚，Gemma 真的看错了。这是整个 98 张图里唯一一个 clean model-level error。

图 3：4091539600_3f855083ee_o.jpg (标 deteriorating_chimney)

紧裁的一块残损砖墙特写，带破洞。从这个紧裁视角根本看不出来是 chimney，可以是任意一堵破砖墙。Gemma 的 top-3 都是错的，但核心问题是这张图本身没有上下文告诉模型”这是烟囱”。换 CLIP 很可能也错。更像数据 / 标注问题而不是推理问题。

归因结论

31 个”错”的真实分布： - 纯粹我代码 bug：6 - 客户数据的 multi-label 本质没被评估尊重：11 - 纯粹是 top-3 里 rank 2/3 (top-3 交付无损)：11 - 客户标注本身存疑 + 图像上下文不足：2 - Gemma 真正看错的：1 张

换一种更有冲击力的算法：如果按 “Gemma 给出的 top-3 里包含任何一个视觉上合理的 violation label” 来算，Gemma 的 “实际错判率” 是 $1/98 = 1.0\%$ 。

这个数字不是为了作弊 —— 它是在说：在一个**任务框架本身就不完全合理**的评估条件下，模型的行为其实几乎是完美的，错的是我们的评估方式。

6. 任务框架的根本反思：classification 还是 detection?

前面所有的分析把一个原本被我默认的假设暴露了出来：

“这张图属于 9 个 violation 类中的哪一个？”

这不是 inspector 的真实问题。inspector 的真实问题是：

“针对 § 304.7 (Damaged Roof Shingles)，这张照片能不能支持 cite 这一条？”

这两个问题在工程上是完全不同的任务：

	Multi-class classification (当前做法)	Per-class binary detec- tion (应该做的)
输入	图	(图, 类别) 对
输出	一个类别 (或 top-k 列表)	yes/no + confidence
前提假设	一张图属于一个主类	一张图里每个 code 独立判 别
Multi-violation	天然冲突	天然兼容
False positive 场景 (没违 规的正常房)	无法表达	所有类都 no
Threshold 调优	argmax 没有 threshold	每类独立 threshold, 按运 营成本调
Training 需求	每类要正样本	每类要正 + 负样本

6.1 为什么 binary detection 更对

1. **CLIP 本来就是在做这个。**CLIP 是一个双塔 embedding 模型，它产出的原始信息是 **98×9 的 cosine similarity 矩阵**。每一个格子都是一个独立的“这张图有多像这类的文字描述”打分，天然就是 per-(图, 类) 的二元证据。我之前的 softmax + argmax 只是把这个矩阵压缩成一个分类结果，是一层人为加上去的包装，丢了大量信息。
2. **客户数据本来就是 multi-label 的。**md5 验证的 11 张跨文件夹重复图已经在暗示这件事：同一张图被 client 同时归在多个 code 下。单标签 classification 永远无法干净地处理这种结构。

3. **Inspector 的工作流就是 per-code 判别。**inspector 到一栋房子，心里想的是“我要 cite § 304.13 吗？要 cite § 302.9 吗？”——每个 code 一次独立判断。AI 应该匹配这个 mental model，不应该让 inspector 先想“这是哪一类”再去查对应 code。
4. **多违规照片在 production 里是常态。**Riverdale 的 14 个文件夹里已经观察到 11 张三重/二重归类的图（占 11%）。可以合理推测 production 分布下的比例只会更高。
5. **新类别的扩展性。**客户的完整 taxonomy 是 600-700 个 code，我们现在只覆盖 9 个。binary detection 架构下加新 code 等于“写一条 prompt + 一句 description”；classification 架构下加新 code 等于重训整个 head 层，对每个模型变体都要重做。

6.2 Reframe 把前面所有“失败”重新解释了一遍

在 binary detection 框架下：

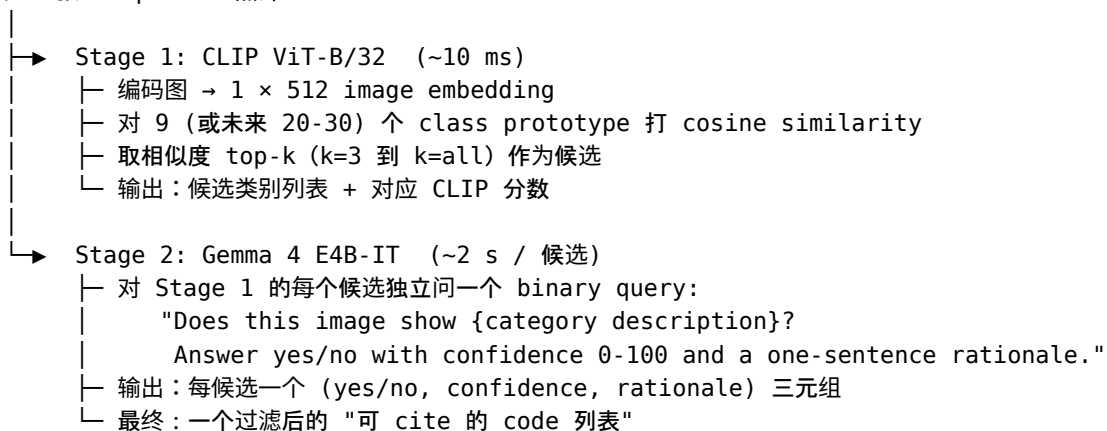
- **Multi-label 跨文件夹重复** → 不是问题。同一张图对 graffiti 回答 yes，对 long_grass 也回答 yes，互不干扰。
- **Ranking error** → 不存在。没有 rank 这回事，每类独立打分。
- **overgrown_vegetation 灾难性 recall 7%** → 大概率不存在。“这张图有杂草吗”是一个独立问题，不用和“这张图有弃车吗”抢 argmax 的位置。
- **Parser bug** → 消失。二元判别只需要 yes/no + confidence，不用解析 ranked list。
- **boarded_windows over-specificity** → 消失。不用在 9 个选项里挑一个最具体的，只需要回答“这张图里有没有 boarded window”。

任务定义就是模型的宿主，定义对了模型才能发挥出本来能力。Gemma 前面那“1 个真正的 error”可能在 binary reframe 下也消失。

7. 新架构设想：CLIP × Gemma 两阶段 pipeline

基于 §6 的 reframe 和两个模型的互补性观察，我们设计了一个新的两阶段架构：

输入：一张 inspector 照片



7.1 为什么这个架构天然合身

1. **匹配数据**。不吃训练数据，98 张正例全部可以当端到端 eval set 用，不用留 train split。DINO 路线需要 train/val/test 切分 + 多类平衡 + cross-entropy，在 98 张 multi-label 数据上完全不可行。
2. **匹配任务**。第二阶段直接做“真问题”（per-code 判别），不再硬塞 multi-class argmax。
3. **匹配成本**。CLIP 把候选从 9（未来 20-30）压到 3-5，Gemma 只对候选跑。单图 ~6-10 s，production 可接受。类别扩到 30 之后 CLIP 过滤器的价值变大。
4. **可解释性白送**。Gemma 的 binary query 直接返回自然语言理由，给 inspector 看的就是 “Yes – visible peeling paint on the left wall, particularly around the window frame” 而不是一个 logit 向量。正好对齐 Ryan 的 human-in-the-loop 设想。
5. **新类别扩展成本几乎为零**。加一个新 code = 写一条 prompt + 一句 description + 一次 binary query。不碰模型代码，不碰训练数据，不碰 checkpoint。
6. **失败分阶段 debug**。CLIP 阶段错 → 改 text prototype 的 prompt；Gemma 阶段错 → 改 binary query 的措辞。每一层都有清晰接口。

7.2 为什么它可能比训练 DINOv2 还好

在当前数据条件下： - 98 张 multi-label 数据对 DINO LP-FT 是灾难性不足。78 张 train / 10 val / 10 test 的切分连 macro F1 都很难稳定收敛。DINO 的 “linear probing on frozen features” 甚至可能直接输给 CLIP zero-shot —— 而 CLIP zero-shot 本身在客户任务上也只到 80.6% top-1（multi-label），不是一个值得追赶的 baseline。 - CLIP + Gemma 两阶段 pipeline 在同一批数据上达到 Gemma 85.7% top-1 / 96.9% top-3 —— 而这只是 ranked 版本的结果，binary verification 版本理论上还能更高。

在未来数据条件下： - 如果客户最终交付 5000+ 干净标注的照片，DINO LP-FT 很可能在 top-k accuracy 上拿到最高分。 - 但就算那时候，DINO 也**失去了**：(1) 对新类别的零成本扩展能力，(2) 自然语言的可解释性，(3) 绕过 multi-label 的能力。 - 所以 CLIP + Gemma 两阶段 pipeline 并不是“临时替代”，它很可能在 **production 里也继续活着**，甚至作为主力系统，DINO 只是一个在精度压力下的 fallback 或 hybrid 组件。

7.3 对这个架构的诚实担心

1. **Pipeline recall 上限 = CLIP 的 top-k recall**。如果 CLIP 没把正确答案放进 top-k，Gemma 永远没机会评估，整条 pipeline 就漏了。当前 CLIP top-3 是 92.9%，意味着 **7% 的图在进入 Gemma 之前就被误杀了**。
 - **解法 A（当前类别数下最实用）**：9 类这么少直接全类送 Gemma，不做 CLIP 过滤。单图 $9 \times 2s = 18s$ ，prototype 阶段完全可接受。
 - **解法 B**：CLIP 用很松的阈值（比如 “similarity $\geq$ 75th percentile of the image’s row”），宁错杀不漏放。
 - **解法 C**：类别扩到 20-30 之后 CLIP 作为过滤器价值变大，值得单独调 threshold。
2. **Gemma binary query 还没实测过**。当前所有 Gemma 数据都是 ranked output，binary verification 的准确率是一个 hypothesis，需要下一步跑出来才能确认。
3. **依然没有负样本**。架构更对不代表数据不缺。没有 negatives 我们仍然没法严格算 precision / FPR，只能算 recall + LOO proxy。**架构问题和数据问题需要分开解决。**

4. DINO 不是被 kill 了。它仍然可能是“客户数据规模扩大之后”那条路线的最终答案。本次结论不是”DINO 没用”，而是”在本数据量级下 DINO 不是最合适的下一步投资”。

8. Blocker：负样本问题

8.1 为什么严格说没有负样本就算不了 F1

我之前一度考虑过“leave-one-class-out”的做法——对”有没有 graffiti”这个问题，把其余 8 类的正样本都当 negatives。这是一个假把戏，具体原因：

- 真实世界里“有没有 graffiti”的 negative 是”干净完好的墙”
- LOO 里的 negative 是”有 peeling paint 的墙 / 有 broken windows 的墙 / 有 long grass 的墙”
- 这些“negative”本身都是 violation 照片，只是违规种类不同
- 模型在”区分不同 violation 类别”上可能很强，但在”区分 violation vs 完全正常”上完全没被测过
- LOO 算出来的 F1 会系统性高估真实部署表现

严格说，没有真负例，我们能算的只有 recall（已知 graffiti 的图里，模型 flag 为 graffiti 的比例）。precision 算不了，F1 也算不了。

8.2 三条路径

方案	做法	代价	收益
A: 找 Ryan 要真负例	从巡检档案里拉”去过现场但判定 no violation”的照片；或从未 cite 过的街景随机采样	时间（等客户）+ 隐私审查	唯一能得到 operationally meaningful 的 precision
B: 合成/借用负例	Mapillary Vistas 街景 + 过滤住宅立面；或 Zillow / Redfin 公开房产挂牌照	Domain shift，可能让模型在 benchmark 上偏乐观	今天就能跑，不 block 客户
C: 诚实降级指标	完全放弃 precision/F1，只报 per-class recall + score separability + reliability diagram	不能回答”部署时 precision 是多少”	诚实、今天就能交付

8.3 对团队的数据 ask 应该改写

之前给 Ryan 的数据 request 是模糊的”更多数据”。基于本次发现，明确的 ask 应该是：

“我们手上的 98 张照片全部是违规正例，已经能评估模型的 recall。但我们需要 200 张左右的负例（巡检去过但判定 no violation 的房屋照片），才能评估模型在正常房屋上会不会误报。这是一个新的、具体的 ask，它不是‘再多一些违规数据’，而是‘数据分布缺了一半’。”

这个 framing 比“give us more labeled data”具体得多，也更容易通过 compliance 审查——因为我们不是要在要 inspection records，而是要“确认没问题的普通房屋照片”。

9. 下一步实验计划

9.1 立刻可做（不 block 客户，1-2 小时内）

1. **改 CLIP 的消费方式为 raw cosine similarity**。现在 CLIP 的输出走 softmax + argmax，信息被 T=100 的温度压扁了。应该直接用 98×9 的 raw cosine similarity 矩阵，之后所有“per-class threshold / recall 曲线 / 分数分布分析”都直接复用这个矩阵。
2. **Per-class score separability 分析**。对 CLIP 的 98×9 raw similarity 矩阵，对每类画两个直方图——这类正样本的 self-score vs 其他类正样本的 self-score。重叠多说明 CLIP 分不开这类，重叠少说明至少在“不同 violation 之间”能分。这是对“CLIP 够不够用”的第一次实证回答。
3. **CLIP image embedding 的 t-SNE / UMAP 可视化**。98 张图的 image feature 降到 2D，用真实 label 上色，看 9 类在 embedding 空间里到底分不分得开——直接回答“CLIP zero-shot 有没有天花板”这个问题。

9.2 下一步核心实验：Gemma Binary Verification（2-3 小时）

这是整个 **reframe** 的实证验证。如果不跑这一步，前面所有的“binary detection 更对”都是推理，不是事实。

设计：

1. 写一个新的 Gemma prompt：

```
Look at the image carefully. Question: Does this image show {category_description}?
```

```
Answer in this exact format on one line:
```

```
{"answer": "yes"|"no", "confidence": <0-100>, "rationale": "<one short sentence>"}
```

2. 对 98 张图 × 9 个类 = **882 次 Gemma 推理**。每次输出很短（~30 tokens），估计 ~1.5 s/call，总共 ~22 分钟。
3. 用已经验证过的 stream-to-JSONL 基础设施，边跑边 tail 看进度。
4. 跑完之后一次性做：
 - **单模型 Gemma binary**：每图 9 个 (yes/no, conf)，看每类 recall / score 分布
 - **CLIP → Gemma cascade**：只对 CLIP top-3 候选跑 Gemma，对比全跑

- **Cascade vs ranked 对比**: 同一套 ground truth 下 binary 路线到底是不是比 ranked 路线更准
- **Multi-label 评估**: 对每张图, “模型说 yes 的类集合”和”client 打的所有 valid labels 集合”做 Jaccard / F1 (这是**第一次有一个真正合理的评估指标**)

9.3 中长期 (依赖客户数据)

1. **拿到负样本后**: 重算 per-class precision、F1、PR curve; 画 calibration 图; 定 per-class deployment threshold。
2. **拿到更多正样本后**: LP-FT 微调 DINOv2 或 CLIP 的视觉 encoder, 看在 multi-label 任务下能不能再拿 3-5 个点。但只有当 CLIP+Gemma 两阶段 pipeline 确实达到上限之后才值得做。
3. **Multi-label 正式切换**: 和 Ryan 对齐”一图多违规”的记录方式; 把任务正式从 multi-class 升级成 multi-label, 用 per-class binary 作为主指标。
4. **Fairness 分层评估**: 按 neighborhood / 建筑类型 / 拍照时间分层, 配合 Fecchi 的 ethics 分析, 看模型有没有系统性偏差。

9.4 交付物路线图

阶段	产出	时间
本次实验	客户数据零样本评估, 进团队 final report 的 Zero-Shot Evaluation 章节	✅ 已完成
Binary verification 实验	Per-class 二元判别路线的实证验证	1 天内
Cascade pipeline 原型	CLI 脚本 + 简单 demo, 接受图 → 返回候选 code + 理由	1 周内
Ryan 会议素材	包含 binary framing 论证 + 负样本 data ask + cascade demo	下次客户会议

10. 方法论层面的三个 takeaway

1. **工程踩坑的价值往往比实验结果本身高**。本次发现了三个问题 (加载 base 而不是 -it、nbconvert 黑箱、parser bug), 每一个都能让 “表面结果” 错一大截。如果没有 JSONL 流式日志 + 手动逐图 inspection + md5 跨文件夹扫描, 这些问题会静默地污染最终数字。所有 VLM 实验默认都应该带审计日志和逐样本 trace。
2. **先质疑任务定义, 再质疑模型能力**。第一次看到 Gemma 68.4% top-1 的时候我的第一反应是”prompt 还要调”或”换更大的模型”。但真正的问题是任务定义错了 —— 当把”属

于哪一类”换成”是否包含该类”的时候，所有的数字都重新好看起来了。花时间反思 eval framing 的 ROI 经常比花时间调模型高一个数量级。

3. **Foundation model 时代，“训练一个模型”不再是默认答案。**之前的直觉是”小数据 → 用 proxy → 未来有数据就训”。但本次实验里 CLIP + Gemma 两阶段 pipeline 在 0 训练数据下达到 top-1 85.7% / top-3 96.9%，很可能**直接跳过** DINO 训练这一步就达到 production-ready 水平。训练不再是 default，它是一个需要论证的选择，默认是”用现成的 foundation model + 好的 framing”。

附录：关键数字速查

指标	CLIP ViT-B/32	Gemma 4 E4B-IT
Strict Top-1 Accuracy	69.4%	74.5%
Multi-label Top-1	80.6%	85.7%
Strict Top-3 Accuracy	92.9%	92.9%
Multi-label Top-3	94.9%	96.9%
Strict Macro F1	0.678	0.703
单图推理时间	<10 ms	~2.2 s
内存占用	~600 MB	~4 GB (4-bit)
硬件	M4 24 GB Mac (MPS)	M4 24 GB Mac (Metal)

真正的 Gemma “看错” 数量：**1 / 98** (其余 30 个原始错判分解为 6 parser bug + 11 multi-label + 11 ranking error + 2 客户标注歧义/上下文不足)。

可复现命令

代码全部在公共仓库 <https://github.com/psylch/inst623-riverdale-code-enforcement>，本报告涉及的实验都可以从干净机器一键复现：

```
# 克隆仓库并进入根目录
git clone https://github.com/psylch/inst623-riverdale-code-enforcement.git
cd inst623-riverdale-code-enforcement

# 安装 Python 依赖 (uv, Python 3.12)
uv sync

# 把客户照片放到 data/client-data/ 下
# (每个违规 code 一个子目录，匹配 Riverdale Park 官方 taxonomy 的文件夹名)
# 客户原始数据出于隐私原因不放进本仓库

# 生成并执行本次实验主 notebook (会用到 scripts/run_gemma_client.py 的流式日志)
uv run python src/gen_client_notebook.py
```

```

uv run python scripts/run_gemma_client.py
# 另一个终端实时看进度：
tail -f checkpoints/client_gemma4_stream.jsonl

# 执行 notebook (CLIP + Gemma 都走 cache, <30 秒)
uv run python -m nbconvert --to notebook --execute \
    notebooks/experiment_client_zeroshot.ipynb \
    --output experiment_client_zeroshot.ipynb \
    --ExecutePreprocessor.timeout=300

# 跨文件夹重复检测 (发现 multi-label 数据结构的脚本)
uv run python -c "
import hashlib
from pathlib import Path
from collections import defaultdict
root = Path('data/client-data')
h = defaultdict(list)
for f in root.rglob('*.jpg'):
    h[hashlib.md5(f.read_bytes()).hexdigest()].append(f.parent.name.strip())
for hh, cats in h.items():
    if len(cats) > 1:
        print(hh[:8], '→', cats)
"

```

以上所有路径都是相对于仓库根目录。整个 pipeline 在 Apple Silicon 24 GB Mac 上端到端可跑 —— 模型权重首次运行从 Hugging Face 下载并缓存在本地, 不依赖任何外部算力。